UNIVERSITÉ DE NAMUR

Econométrie

Nettoyage des données dans R

Jérémy Do Nascimento Miguel
Université de Namur
2026-10-08

Disclaimer

1. Cette session a été développé en collaboration avec Mathilde Col, initiallement pour un cours à Addis Ababa University.



Rappel de la séance précédente

- R : le langage de programmation, le logiciel interpréteur, la console
- RStudio : l'IDE pour la console, les scripts, l'environnement, les graphiques et plus
- Organiser un projet
- Les packages : installer et charger
- Types et opérateurs
- Structures de données : vecteurs, matrices et data frames
- Importer des données dans R
- Sous-ensembles (*subsetting*) de variables et d'observations



Cette séance

Nettoyage de base et économétrie avec des données réelles dans R :

- Nettoyer les données avec le package dplyr
- Introduction aux expressions régulières (`regex`)
- Fonctions & boucles
- Fusionner et empiler deux jeux de données
- Remodeler un jeu de données (`reshape`)
- Statistiques descriptives



Nettoyage des données avec `dplyr`

Trier les données

- En base R, vous pouvez utiliser la fonction `order`

```
mydatas3[order(mydatas3$num_deaths), ]
```



Trier les données

- En base R, vous pouvez utiliser la fonction `order`

```
mydatas3[order(mydatas3$num_deaths), ]
```

- Utilisez la fonction `arrange()` du package `dplyr` pour trier selon les valeurs d'une variable spécifique

```
arrange(mydatas3, num_deaths)
```



Trier les données

- En base R, vous pouvez utiliser la fonction `order`

```
mydatas3[order(mydatas3$num_deaths), ]
```

- Utilisez la fonction `arrange()` du package `dplyr` pour trier selon les valeurs d'une variable spécifique

```
arrange(mydatas3, num_deaths)
```

- Par défaut, le tri est croissant (les observations les plus anciennes en haut ici)



Trier les données

- En base R, vous pouvez utiliser la fonction `order`

```
mydatas3[order(mydatas3$num_deaths), ]
```

- Utilisez la fonction `arrange()` du package `dplyr` pour trier selon les valeurs d'une variable spécifique

```
arrange(mydatas3, num_deaths)
```

- Par défaut, le tri est croissant (les observations les plus anciennes en haut ici)
- Pour un tri décroissant, ajoutez un signe moins devant la variable

```
mydatas3[order(-mydatas3$num_deaths), ]
arrange(mydatas3, -num_deaths)
```



Modifier les données

Avec `dplyr`, vous disposez de la fonction `mutate()`

```
mydatas4 <- mydatas3 %>%
    mutate(
    initial_pop = total_population + num_deaths
    )
```



Modifier les données

Avec `dplyr`, vous disposez de la fonction `mutate()`

```
mydatas4 <- mydatas3 %>%
    mutate(
    initial_pop = total_population + num_deaths
    )
```

La fonction `mutate()` permet de faire plusieurs modifications à la fois

```
mydatas4 <- mydatas3 %>%
    mutate(
    initial_pop = total_population + num_deaths,
    mort_rate = num_deaths / total_population
    )
```



Modifier un sous-ensemble des données

Avec `dplyr`, il faut combiner `mutate()` avec la fonction `case_when()`

```
mydatas4 <- mydatas3 %>%
    mutate(
        any_death = case_when(
        num_deaths == 0 ~  "aucun",
        num_deaths >  0 ~ "morts",
        .default = NA
    )
)
```



Modifier un sous-ensemble des données

Avec `dplyr`, il faut combiner `mutate()` avec la fonction `case_when()`

```
mydatas4 <- mydatas3 %>%
    mutate(
        any_death = case_when(
        num_deaths == 0 ~  "aucun",
        num_deaths >  0 ~ "morts",
        .default = NA
    )
)
```

Vous pouvez aussi combiner `mutate()` avec la fonction `ifelse()` (de base R) s'il n'y a qu'une seule condition :

```
mydatas4 <- mydatas3 %>%
    mutate(
        any_death = ifelse(num_deaths > 0, "aucun", "morts")
    )
```



Modifier par groupe

Vous pouvez utiliser la fonction `group_by()` :

```
mydatas3_grouped <- mydatas3 %>%
    group_by(state) %>%
    mutate(state_mort_rate = sum(num_deaths)/sum(total_population))
```

- Cela donne la même valeur pour toutes les observations de chaque état
- Pour dégrouper les données, utilisez la fonction `ungroup()`
- Depuis dplyr 1.1.0 en 2023, l'option .by a été ajoutée

```
mydatas3_grouped <- mydatas3 %>%
    mutate(state_mort_rate = sum(num_deaths)/sum(total_population),
            .by = state)
```



Contraintes sur les types de données

Type de donnée	Exemple	Type R
Texte	Pays, ville, nom, adresse	character
Entier	Population, main-d'œuvre, nombre de pièces	integer, numeric
Décimal	Température, PIB	numeric, double
Binaire/Dummy	Femme/Homme, Marié/Célibataire	logical, factor
Catégorie	Statut marital, fuseau horaire	factor
Date	Date de naissance, date de commande	Date

Contraintes sur les types de données

Type de donnée	Exemple	Type R
Texte	Pays, ville, nom, adresse	character
Entier	Population, main-d'œuvre, nombre de pièces	integer, numeric
Décimal	Température, PIB	numeric, double
Binaire/Dummy	Femme/Homme, Marié/Célibataire	logical, factor
Catégorie	Statut marital, fuseau horaire	factor
Date	Date de naissance, date de commande	Date

- Chacun de ces types de données est traité différemment
- Nous devons nous assurer que chaque variable de notre jeu de données a le bon type



Vérifier les types de données

Vérification logique : renvoie TRUE ou FALSE

- `is.character()`, `is.numeric()`, `is.logical()`, `is.factor()`, `is.Date()`, ...



Vérifier les types de données

Vérification logique : renvoie TRUE ou FALSE

- `is.character()`, `is.numeric()`, `is.logical()`, `is.factor()`, `is.Date()`, ...

En écrivant le code, on effectue aussi des vérifications manuelles avec `table()` ou `summary()` :

- l'âge est supérieur à 0 et inférieur à 100
- la date de naissance précède la date de décès
- les noms de pays fournis figurent dans la liste des noms de pays valides
- la somme des sous-groupes est égale au total
- la valeur moyenne est proche des statistiques officielles
- ...



Transformer des variables

- On peut utiliser les fonctions `as.*()` pour transformer une variable d'un type à un autre

```
mydatas3$cases <- as.numeric(mydatas3$cases)
class(mydatas3$case)
# [1] "numeric"
```



Transformer des variables

- On peut utiliser les fonctions `as.*()` pour transformer une variable d'un type à un autre

```
mydatas3$cases <- as.numeric(mydatas3$cases)
class(mydatas3$case)
# [1] "numeric"
```

- On peut maintenant estimer le nombre moyen de cas

```
mean(mydatas3$cases)
# [1] 40.52027
```



Nettoyer les chaînes de caractères

Nous allons voir comment utiliser le package stringi et sa version simplifiée, le package `stringr`.



Nettoyer les chaînes de caractères

Nous allons voir comment utiliser le package stringi et sa version simplifiée, le package `stringr`.

Première manipulation : mettre en minuscules, MAJUSCULES, Mettre en Majuscule Chaque Mot (documentation Stringr)

```
library(stringr)
str_to_lower("Hello world")
# [1] "hello world"
str_to_upper("Hello world")
# [1] "HELLO WORLD"
str_to_title("Hello world")
# [1] "Hello World"
```



Le problème des virgules

- Examinons la variable *percent_below_poverty*

```
class(mydatas3$percent_below_poverty)
# [1] "character"
head(mydatas3$percent_below_poverty)
# [1] "9,3" "9,3" "9,3" "9,3" "9,3" "9,3"
```



Le problème des virgules

- Examinons la variable *percent_below_poverty*

```
class(mydatas3$percent_below_poverty)
# [1] "character"
head(mydatas3$percent_below_poverty)
# [1] "9,3" "9,3" "9,3" "9,3" "9,3" "9,3"
```

- On a des virgules à la place des points pour indiquer les décimales

```
as.numeric(mydatas3$percent_below_poverty)
```



Le problème des virgules

- Examinons la variable *percent_below_poverty*

```
class(mydatas3$percent_below_poverty)
# [1] "character"
head(mydatas3$percent_below_poverty)
# [1] "9,3" "9,3" "9,3" "9,3" "9,3" "9,3"
```

- On a des virgules à la place des points pour indiquer les décimales

```
as.numeric(mydatas3$percent_below_poverty)
```

- R remplacera par NA les observations contenant des virgules



Un peu d'expressions régulières

- Il faut remplacer les virgules par des points AVANT de transformer en numérique
- Le package `stringr` est une version simplifiée du package `stringi`

```
library(stringr)
str_replace(mydatas3$percent_below_poverty, ",", ".")
# str_replace_all() remplacera TOUTES les occurences
```



Un peu d'expressions régulières

- Il faut remplacer les virgules par des points AVANT de transformer en numérique
- Le package `stringr` est une version simplifiée du package `stringi`

```
library(stringr)
str_replace(mydatas3$percent_below_poverty, ",", ".")
# str_replace_all() remplacera TOUTES les occurences
```

- On peut maintenant remplacer la chaîne de caractères par un nombre

```
mydatas3$percent_below_poverty <- mydatas3$percent_below_poverty |>
    str_replace(",", ".") |>
    as.numeric()
```



Encore des expressions régulières

Trouver et remplacer des motifs (*patterns*) dans des chaînes de caractères

```
str_detect(c("apple", "banana", "pear"), pattern = "an")
# [1] FALSE TRUE FALSE
```

- . : n'importe quel caractère
- ? : le caractère précédent est optionnel
-
  - : au moins une occurrence du caractère précédent
-
  - : 0 ou plusieurs occurrences du caractère précédent
- {a, b} : entre a et b occurrences du caractère précédent
- [...] : n'importe lequel des caractères entre crochets
- | : un motif ou un autre
- ^ : début de chaîne
- \$: fin de chaîne
- \ : échapper les caractères spéciaux

```
str_extract("Born the 01/01/2000 in Addis", "[0-9]{2}/[0-9]{2}/[0-9]{4}")
# [1] "01/01/2000"
```



Encodage des chaînes de caractères

- Dans un ordinateur, tout est stocké sur 8 bits : un nombre entre 0 et 255
- encodage : chaque nombre est associé à une valeur



Encodage des chaînes de caractères

- Dans un ordinateur, tout est stocké sur 8 bits : un nombre entre 0 et 255
- encodage : chaque nombre est associé à une valeur
- Le plus basique est l'encodage ASCII, qui couvre la plupart des caractères
- Aujourd'hui, on utilise principalement l'UTF-8 pour la majorité des documents



Encodage des chaînes de caractères

- Dans un ordinateur, tout est stocké sur 8 bits : un nombre entre 0 et 255
- encodage : chaque nombre est associé à une valeur
- Le plus basique est l'encodage ASCII, qui couvre la plupart des caractères
- Aujourd'hui, on utilise principalement l'UTF-8 pour la majorité des documents
- Vous pourrez rencontrer, ou devoir utiliser, d'autres encodages avec certains fichiers, en particulier ceux utilisant d'autres alphabets comme le chinois, le japonais, le cyrillique ou l'arabe



Encodage des chaînes de caractères

- Dans un ordinateur, tout est stocké sur 8 bits : un nombre entre 0 et 255
- encodage : chaque nombre est associé à une valeur
- Le plus basique est l'encodage ASCII, qui couvre la plupart des caractères
- Aujourd'hui, on utilise principalement l'UTF-8 pour la majorité des documents
- Vous pourrez rencontrer, ou devoir utiliser, d'autres encodages avec certains fichiers, en particulier ceux utilisant d'autres alphabets comme le chinois, le japonais, le cyrillique ou l'arabe
- Lors de la lecture ou de l'écriture d'un fichier, vous avez en général une option *encoding*



Encodage des chaînes de caractères

- Dans un ordinateur, tout est stocké sur 8 bits : un nombre entre 0 et 255
- encodage : chaque nombre est associé à une valeur
- Le plus basique est l'encodage ASCII, qui couvre la plupart des caractères
- Aujourd'hui, on utilise principalement l'UTF-8 pour la majorité des documents
- Vous pourrez rencontrer, ou devoir utiliser, d'autres encodages avec certains fichiers, en particulier ceux utilisant d'autres alphabets comme le chinois, le japonais, le cyrillique ou l'arabe
- Lors de la lecture ou de l'écriture d'un fichier, vous avez en général une option *encoding*
- Astuce utile pour retirer les accents avec le package **stringi** : `stri_trans_general`



Supprimer les doublons

- Les doublons sont un problème courant dans les données brutes d'enquête
- On peut utiliser la fonction `distinct()` du package `dplyr`

```
data_unique <- distinct(mydatas3)
sum(duplicated(data_unique))
```

- *data_unique* ne contient en effet plus aucun doublon



Valeurs manquantes

- Idéalement enregistrées comme NA
- Mais souvent notées "NA", -1, 0, 8, 99, ... dans les données
- Vous pouvez utiliser la fonction `is.na()` pour repérer les NA

```
is.na(miss_rain)
```



Valeurs manquantes

- Idéalement enregistrées comme NA
- Mais souvent notées "NA", -1, 0, 8, 99, ... dans les données
- Vous pouvez utiliser la fonction `is.na()` pour repérer les NA

```
is.na(miss_rain)
```

- Vous pouvez les compter en sommant le résultat avec la fonction `sum()`

```
sum(is.na(miss_rain))
# [1] 100849
```



Valeurs manquantes

- Idéalement enregistrées comme NA
- Mais souvent notées "NA", -1, 0, 8, 99, ... dans les données
- Vous pouvez utiliser la fonction `is.na()` pour repérer les NA

```
is.na(miss_rain)
```

- Vous pouvez les compter en sommant le résultat avec la fonction `sum()`

```
sum(is.na(miss_rain))
# [1] 100849
```

L'imputation la plus basique consiste à remplacer une valeur manquante par la moyenne, en utilisant la fonction `replace_na()` du package `tidyr`

```
data_replace_3 <- data_unique |>
    mutate(rain_filled = replace_na(rain, mean(rain, na.rm = TRUE)))
```



Tâche 1

05:00

- Ouvrez le jeu de données "hh_characteristic.xlsx" et stockez-le sous le nom de votre choix. Il contient des informations sur
- Nettoyez la variable "sex" pour qu'elle prenne la valeur 0 pour les hommes et 1 pour les femmes
- Créez la variable "educ" à partir de la variable "edyears" pour qu'elle corresponde à la catégorie :
  - 0 = Aucune éducation
  - 1 = Primaire non complété
  - 2 = Primaire complété
  - 3 = Secondaire non complété
  - 4 = Secondaire complété
  - 5 = Tertiaire
- Nettoyez le nom du woreda (admin3name_en) : enlever les under-score et les "at" dans les noms



Fonctions & Boucles

Fonctions

- Vous pouvez écrire votre propre fonction, la stocker dans un objet et la réutiliser plus tard
- Une fonction est composée des **arguments** et du **corps/code** de la fonction

```
ma_fonction <- function(arguments) {
    # Code R
}
```



Fonctions

- Vous pouvez écrire votre propre fonction, la stocker dans un objet et la réutiliser plus tard
- Une fonction est composée des **arguments** et du **corps/code** de la fonction

```
ma_fonction <- function(arguments) {
    # Code R
}
```

Un exemple simple

```
addition <- function(a, b) {
    c <- a + b
    return(c)
}
addition(2, 5)
# [1] 7
```



Boucles : boucles for

- Les boucles sont une structure de programmation très importante
- Comme son nom l'indique, dans une boucle, le programme répète un ensemble d'instructions jusqu'à ce qu'une condition soit remplie
- La syntaxe d'une boucle `for` est

```
for (x in liste) {
    # Code R à exécuter
}
```



Boucles for : exemple 1

Boucler sur un vecteur

```
fruits <- c("mangues", "bananes", "kiwis")
for (fruit in fruits) {
    # La fonction paste() concatène des chaînes de caractères
    print(
        paste("J'aime les", fruit)
    )
}
# [1] "J'aime les mangues"
# [1] "J'aime les bananes"
# [1] "J'aime les kiwis"
```

La fonction `paste()` est une fonction de base pour concaténer des chaînes de caractères



Boucles for : exemple 2

Boucles imbriquées

```
for (i in 2:3) {
    for (fruit in fruits) {
        print(
            paste("Je peux avoir", i, fruit, "s'il vous plaît ?")
        )
    }
}
# [1] "Je peux avoir 2 mangues s'il vous plaît ?"
# [1] "Je peux avoir 2 bananes s'il vous plaît ?"
# [1] "Je peux avoir 2 kiwis s'il vous plaît ?"
# [1] "Je peux avoir 3 mangues s'il vous plaît ?"
# [1] "Je peux avoir 3 bananes s'il vous plaît ?"
# [1] "Je peux avoir 3 kiwis s'il vous plaît ?"
```



Boucles : la famille apply

- Une autre façon de faire des boucles est la famille de fonctions `apply()`
- Elles *appliquent* une fonction donnée à tous les éléments d'une liste
- La plus utilisée est `lapply()`, qui renvoie une liste de résultats

```
lapply(liste, fonction)
```



Boucles : la famille apply

- Une autre façon de faire des boucles est la famille de fonctions `apply()`
- Elles *appliquent* une fonction donnée à tous les éléments d'une liste
- La plus utilisée est `lapply()`, qui renvoie une liste de résultats

```
lapply(liste, fonction)
```

Un court exemple

```
# Applique la fonction racine carrée à tous les nombres de 1 à 10
lapply(1:10, sqrt)
```



Boucles : la famille apply

- Une autre façon de faire des boucles est la famille de fonctions `apply()`
- Elles *appliquent* une fonction donnée à tous les éléments d'une liste
- La plus utilisée est `lapply()`, qui renvoie une liste de résultats

```
lapply(liste, fonction)
```

Un court exemple

```
# Applique la fonction racine carrée à tous les nombres de 1 à 10
lapply(1:10, sqrt)
```

Vous pouvez ajouter des paramètres à une fonction, ou définir une fonction courte qui prend un paramètre

```
# Applique la fonction sprintf() de 1 à 10, avec 2 chiffres et un 0 devant
lapply(1:10, sprintf, fmt = "%02d")
# Applique une fonction qui prend un paramètre x et renvoie x / (1 + x)
# À tous les nombres de 1 à 10
lapply(1:10, function(x) x / (1 + x))
```



Remodeler les données (`reshape`)

Tableau croisé : de large à long (*wide to long*)

La syntaxe de `pivot_longer()` repose sur 4 éléments

- data : les données à pivoter
- cols : la liste des colonnes contenant les valeurs à regrouper
- names_to : la nouvelle colonne à créer pour stocker les noms de ces colonnes correspondant aux valeurs
- values_to : la nouvelle colonne à créer pour stocker les valeurs de ces colonnes



Tableau croisé : de large à long, exemple

- En utilisant les données de population de la Banque mondiale du package `tidyr`
- Vous pouvez accéder aux données de la Banque mondiale avec le package `wbstat`

```
data("world_bank_pop")
world_bank_pop
```

On a :

- Une ligne par pays/indicateur
- Une colonne par année
- Les cellules sont les valeurs

On veut d'abord :

- Une ligne par pays/année/indicateur
- Une nouvelle colonne pour identifier l'année
- Une nouvelle colonne avec les valeurs

```
world_bank_pop_long <- world_bank_pop |>
  pivot_longer(
    cols = `2000`:`2017`,
    names_to = "year",
    values_to = "value"
  )
```



Tableau croisé : de long à large (*long to wide*)

La syntaxe de `pivot_wider()` repose sur 3 éléments

- `data` : les données à pivoter
- `names_from` : la colonne dont les valeurs serviront de noms aux nouvelles colonnes
- `values_from` : la colonne avec les valeurs qui rempliront ces nouvelles colonnes



Tableau croisé : de long à large, exemple

- En utilisant la version longue des données de la Banque mondiale
- La structure de ces données est :
  - Une ligne par pays, année et indicateur
  - Une colonne avec toutes les valeurs
  - Les cellules sont les valeurs
- On veut :
  - Une ligne par pays/année
  - Une colonne pour identifier pays/année
  - Une colonne par indicateur

```
world_bank_pop_clean <- world_bank_pop_long |>
  pivot_wider(
    names_from = "indicator",
    values_from = "value"
  )
```



- L'exemple dans la documentation est un peu différent, je vous laisse le consulter

Combiner des bases de données

- En général, vous utilisez plusieurs bases de données que vous combinez
- Empiler des bases de données avec les mêmes colonnes est simple avec `rbind()`

```
mydatas3_rows_01_10 <- mydatas3[1:10, ]
mydatas3_rows_20_30 <- mydatas3[20:30, ]
rbind(mydatas3_rows_01_10, mydatas3_rows_20_30)
```



Combiner des bases de données

- En général, vous utilisez plusieurs bases de données que vous combinez
- Empiler des bases de données avec les mêmes colonnes est simple avec `rbind()`

```
mydatas3_rows_01_10 <- mydatas3[1:10, ]
mydatas3_rows_20_30 <- mydatas3[20:30, ]
rbind(mydatas3_rows_01_10, mydatas3_rows_20_30)
```

- Fusionner des bases de données avec les mêmes lignes est aussi simple avec `cbind()`

```
mydatas3_cols_1_2 <- mydatas3[, 1:2]
mydatas3_cols_3_4 <- mydatas3[, 3:4]
cbind(mydatas3_cols_1_2, mydatas3_cols_3_4)
```



Combiner des bases de données

- En général, vous utilisez plusieurs bases de données que vous combinez
- Empiler des bases de données avec les mêmes colonnes est simple avec `rbind()`

```
mydatas3_rows_01_10 <- mydatas3[1:10, ]
mydatas3_rows_20_30 <- mydatas3[20:30, ]
rbind(mydatas3_rows_01_10, mydatas3_rows_20_30)
```

- Fusionner des bases de données avec les mêmes lignes est aussi simple avec `cbind()`

```
mydatas3_cols_1_2 <- mydatas3[, 1:2]
mydatas3_cols_3_4 <- mydatas3[, 3:4]
cbind(mydatas3_cols_1_2, mydatas3_cols_3_4)
```

- Cependant, en général vous n'avez pas des données parfaitement alignées à fusionner
- Vous voulez fusionner les jeux de données en fonction de leurs relations



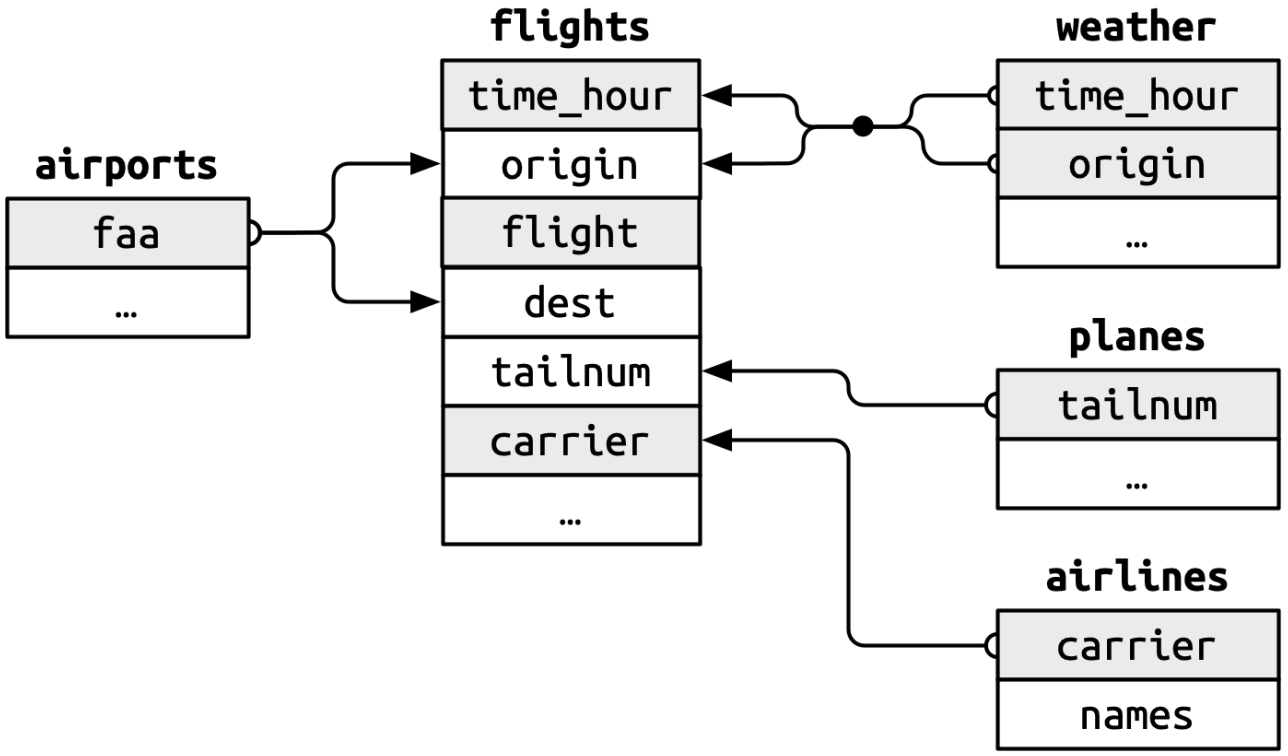
Joindre des données dans R

- Dans R, cela peut se faire avec **base** R ou le package `dplyr`
- Le chapitre de R for Data Science sur les jointures entre en détail avec `dplyr`
- Avec **base** R, vous pouvez utiliser la fonction `merge()`
- Nous utiliserons le groupe de bases de données sur les vols du package `nycflights13`
  - `airlines` : la liste des compagnies aériennes identifiées par un code transporteur (*carrier*)
  - `airports` : la liste des aéroports identifiés par un code à trois lettres (*faa*)
  - `planes` : la liste des avions identifiés par leur numéro de queue (*tailnum*)
  - `weather` : relevés météo dans les aéroports identifiés par leur aéroport et l'heure (*origin* et *time_hour*)
  - `flights` : la liste des vols et leurs caractéristiques

```
data("airlines", package = "nycflights13")
data("airports", package = "nycflights13")
data("planes", package = "nycflights13")
data("weather", package = "nycflights13")
data("flights", package = "nycflights13")
```



Résumé des données de vols



- On garde certaines colonnes des données

```
flights2 <- select(flights, year, time_hour, origin, dest,
    tailnum, carrier)
```

Types de jointures

- Différents types de fusion/jointure :
  - `left join` : garde toutes les valeurs de la première table, et ajoute des valeurs manquantes de la seconde table quand aucune correspondance n'est trouvée
  - `right join` : garde toutes les valeurs de la seconde table, et ajoute des valeurs manquantes de la première table quand aucune correspondance n'est trouvée
  - `inner join` : garde uniquement les valeurs des clés présentes dans les deux tables
  - `outer join` : garde toutes les valeurs, et ajoute des valeurs manquantes des deux côtés si nécessaire



Types de jointures

- Différents types de fusion/jointure :
  - `left join` : garde toutes les valeurs de la première table, et ajoute des valeurs manquantes de la seconde table quand aucune correspondance n'est trouvée
  - `right join` : garde toutes les valeurs de la seconde table, et ajoute des valeurs manquantes de la première table quand aucune correspondance n'est trouvée
  - `inner join` : garde uniquement les valeurs des clés présentes dans les deux tables
  - `outer join` : garde toutes les valeurs, et ajoute des valeurs manquantes des deux côtés si nécessaire

- En **base** R, vous pouvez utiliser la fonction `merge()` pour faire une jointure interne (*inner join*) par défaut, avec les options `all.x` et `all.y` pour choisir si vous voulez garder les non-correspondances de l'un ou l'autre jeu de données

- Avec **dplyr**, vous pouvez utiliser les fonctions `left_join()`, `right_join()`, `inner_join()` et `full_join()`



Exemples de jointures

- Avec la fonction `merge()`

```
merge(flights2, airlines)
```



Exemples de jointures

- Avec la fonction `merge()`

```
merge(flights2, airlines)
```

- Avec la fonction `left_join()`

```
left_join(flights2, airlines)
# Joining with `by = join_by(carrier)`
# A tibble: 336,776 × 7...
```

- `left_join()` est plus explicite, plus rapide, et vous indique quelle colonne est utilisée
- Idéalement, vous devriez préciser la colonne utilisée avec l'option **by**

```
merge(flights2, airlines, by = "carrier")
left_join(flights2, airlines, by = join_by(carrier))
```



Exemples de jointures

- Avec la fonction `merge()`

```
merge(flights2, airlines)
```

- Avec la fonction `left_join()`

```
left_join(flights2, airlines)
# Joining with `by = join_by(carrier)`
# A tibble: 336,776 × 7...
```

- `left_join()` est plus explicite, plus rapide, et vous indique quelle colonne est utilisée
- Idéalement, vous devriez préciser la colonne utilisée avec l'option **by**

```
merge(flights2, airlines, by = "carrier")
left_join(flights2, airlines, by = join_by(carrier))
```

- Si les colonnes ont des noms différents, vous pouvez utiliser les options `by.x` et `by.y` pour `merge()`, ou le == dans la fonction `join_by()` pour `left_join()`



Tâche 2

05:00

Utilisez le jeu de données de l'exercice 1 :

- Vous voulez 1 ménage = 1 observation, comment faites-vous ?
  - Identifiez l'identifiant unique par observation/ligne dans la version longue
  - Passez les données de long à large (*wide*)
- Vous voulez fusionner le jeu de données obtenu avec les données géographiques utilisées lors de la séance 1 :
  - Ouvrez le jeu de données géographiques avec les informations de population
  - Identifiez la variable commune entre les deux, et vérifiez sa compatibilité dans les deux jeux de données
  - Fusionnez les deux et stockez le résultat dans un nouveau jeu de données



Statistiques descriptives

Statistiques de base pour l'analyse de données

Ce que l'on vérifie généralement avant de commencer une analyse de régression :

- Taille de l'échantillon
- Valeurs moyennes : moyenne, médiane (globale et par groupe)
- Dispersion des données : min, max, écart-type
- Distribution : percentiles, asymétrie (*skewness*), histogramme
  - `quantile()` pour obtenir le quantile, `percent_rank(x)` pour savoir dans quel percentile se situe l'observation, `cum_dist()` pour obtenir la distribution cumulative d'une variable
- Fréquence des valeurs
  - On utilise `table()` pour obtenir la fréquence et les valeurs de chaque variable



Statistiques de base pour l'analyse de données

- Nombre d'observations (lignes) et de variables (colonnes)

```
nrow(mydata)
ncol(mydata)
```



Statistiques de base pour l'analyse de données

- Nombre d'observations (lignes) et de variables (colonnes)

```
nrow(mydata)
ncol(mydata)
```

- Statistiques descriptives de base pour toutes les variables, ou une variable spécifique

```
# Pour toutes les variables des données
summary(mydata)
# Pour la variable tracking uniquement
summary(mydata$tracking)
```



Statistiques de base pour l'analyse de données

- Nombre d'observations (lignes) et de variables (colonnes)

```
nrow(mydata)
ncol(mydata)
```

- Statistiques descriptives de base pour toutes les variables, ou une variable spécifique

```
# Pour toutes les variables des données
summary(mydata)
# Pour la variable tracking uniquement
summary(mydata$tracking)
```

- Tabulations croisées à deux variables

```
table(mydata$tracking, mydata$etpteacher)
```



Statistiques agrégées

Si vos données sont au format long, vous pouvez utiliser `group_by(*variable de groupement*)` ou `.by = "*variable de groupement*"` :

```
dat1 <- df_long %>%
    mutate(mean_gender = mean(gender, na.rm = T), .by = hhid)
dat1 <- df_long %>% group_by(hhid) %>%
    mutate(mean_gender = mean(gender, na.rm = T))
```



Statistiques agrégées

Si vos données sont au format long, vous pouvez utiliser `group_by(*variable de groupement*)` ou `.by = "*variable de groupement*"` :

```
dat1 <- df_long %>%
    mutate(mean_gender = mean(gender, na.rm = T), .by = hhid)
dat1 <- df_long %>% group_by(hhid) %>%
    mutate(mean_gender = mean(gender, na.rm = T))
```

Si vos données sont au format large (*wide*), vous pouvez utiliser `rowMeans()` :

```
dat1 <- df_wide %>%
    mutate(mean_all = rowMeans(.),
        mean_sel = rowMeans(.[select_vars]))
```



Statistiques agrégées

Si vos données sont au format long, vous pouvez utiliser `group_by(*variable de groupement*)` ou `.by = "*variable de groupement*"` :

```
dat1 <- df_long %>%
    mutate(mean_gender = mean(gender, na.rm = T), .by = hhid)
dat1 <- df_long %>% group_by(hhid) %>%
    mutate(mean_gender = mean(gender, na.rm = T))
```

Si vos données sont au format large (*wide*), vous pouvez utiliser `rowMeans()` :

```
dat1 <- df_wide %>%
    mutate(mean_all = rowMeans(.),
        mean_sel = rowMeans(.[select_vars]))
```

Autres commandes utiles :

- `n()` compte le nombre d'observations
- `sum()` calcule la somme d'une variable



Tâche 3

05:00

Utilisez le jeu de données produit à l'exercice 2 :

- OPTIONNEL MAIS FACILITERA LA SUITE DES EXERCICES : repassez les données de large à long
- Quelle est la part de femmes dans l'échantillon ?
- Quelle est la taille moyenne des ménages ?
- Quelle est la part de la population interrogée dans chaque woreda (pour mille) ?

UNIVERSITÉ DE NAMUR

BONNE JOURNÉE !

	jeremy.donascimento@unamur.be
	Discord
	Slides